Lab Assignment - 8

Name : Om Tambe **Roll No.** 49

Code :

```
#include <stdio.h>
#include <stdlib.h>

// Structure of a BST node
struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

// Create a new node
struct Node* createNode(int data) {
    struct Node* newNode =
        (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

// Insert a node into the BST
struct Node* insert(struct Node* root, int data) {
    if (root == NULL)
        return createNode(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else if (data > root->data)
        root->right = insert(root->right, data);
    else
        printf("Duplicate values are not allowed.\n");
```

```
    return root;
}

// Inorder traversal: Left, Root, Right
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

// Preorder traversal: Root, Left, Right
void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

// Postorder traversal: Left, Right, Root
void postorder(struct Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

// Count nodes in the longest path from root
int longestPath(struct Node* root) {
    if (root == NULL)
        return 0;

    int leftHeight = longestPath(root->left);
    int rightHeight = longestPath(root->right);

    if (leftHeight > rightHeight)
        return leftHeight + 1;
    else
        return rightHeight + 1;
}

// Find the minimum value in the BST
struct Node* findMin(struct Node* root) {
    if (root == NULL)
        return NULL;
```

```
    while (root->left != NULL)
        root = root->left;

    return root;
}

// Search for a value in the BST
struct Node* search(struct Node* root, int key) {
    if (root == NULL || root->data == key)
        return root;

    if (key < root->data)
        return search(root->left, key);
    else
        return search(root->right, key);
}

// Mirror the tree by swapping left and right pointers
void mirror(struct Node* root) {
    if (root == NULL)
        return;

    struct Node* temp = root->left;
    root->left = root->right;
    root->right = temp;

    mirror(root->left);
    mirror(root->right);
}

// Free allocated memory
void freeTree(struct Node* root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main() {
    struct Node* root = NULL;
    struct Node* result;
    int choice, value, n, i;

    printf("Enter number of initial nodes: ");
    if (scanf("%d", &n) != 1 || n < 0) {
        printf("Invalid input!\n");
```

```
    return 1;
}

for (i = 0; i < n; i++) {
    printf("Enter value: ");

    if (scanf("%d", &value) != 1) {
        printf("Invalid input!\n");
        freeTree(root);
        return 1;
    }

    root = insert(root, value);
}

do {
    printf("\n\n===== BST OPERATIONS =====\n");
    printf("1. Insert a new node\n");
    printf("2. Display inorder traversal\n");
    printf("3. Display preorder traversal\n");
    printf("4. Display postorder traversal\n");
    printf("5. Find longest path from root\n");
    printf("6. Find minimum value\n");
    printf("7. Mirror the tree\n");
    printf("8. Search a value\n");
    printf("9. Exit\n");
    printf("Enter your choice: ");

    if (scanf("%d", &choice) != 1) {
        printf("Invalid input!\n");
        break;
    }

    switch (choice) {
        case 1:
            printf("Enter value to insert: ");
            if (scanf("%d", &value) != 1) {
                printf("Invalid input!\n");
                freeTree(root);
                return 1;
            }
            root = insert(root, value);
            break;

        case 2:
            printf("Inorder traversal: ");
            inorder(root);
            printf("\n");
```

```
            break;

        case 3:
            printf("Preorder traversal: ");
            preorder(root);
            printf("\n");
            break;

        case 4:
            printf("Postorder traversal: ");
            postorder(root);
            printf("\n");
            break;

        case 5:
            printf("Nodes in longest path: %d\n",
                   longestPath(root));
            break;

        case 6:
            result = findMin(root);
            if (result == NULL)
                printf("Tree is empty.\n");
            else
                printf("Minimum value: %d\n",
                       result->data);
            break;

        case 7:
            mirror(root);
            printf("Tree mirrored successfully.\n");
            printf("Inorder traversal after mirroring: ");
            inorder(root);
            printf("\n");
            break;

        case 8:
            printf("Enter value to search: ");
            if (scanf("%d", &value) != 1) {
                printf("Invalid input!\n");
                freeTree(root);
                return 1;
            }

            result = search(root, value);

            if (result != NULL)
                printf("%d found in the tree.\n", value);
```

```
            else
                printf("%d not found in the tree.\n",
                    value);
            break;

        case 9:
            printf("Exiting program...\n");
            break;

        default:
            printf("Invalid choice! Try again.\n");
      }

   } while (choice != 9);

   freeTree(root);

   return 0;
}
```

Output :

```
Enter number of initial nodes: 3
Enter value: 10
Enter value: 20
Enter value: 30


===== BST OPERATIONS =====
1. Insert a new node
2. Display inorder traversal
3. Display preorder traversal
4. Display postorder traversal
5. Find longest path from root
6. Find minimum value
7. Mirror the tree
8. Search a value
9. Exit
Enter your choice: 2
Inorder traversal: 10 20 30


===== BST OPERATIONS =====
1. Insert a new node
2. Display inorder traversal
3. Display preorder traversal
4. Display postorder traversal
5. Find longest path from root
6. Find minimum value
7. Mirror the tree
8. Search a value
9. Exit
Enter your choice:
```